Introduction to Machine Learning and Regression Problems

Marco Laudato – KTH Department of Engineering Mechanics

MOGE 2025, Paris (FR), 18 June 2025

Overview of the lectures

We will cover the following curriculum:

(Github repo: https://github.com/MarcoLaud/VP2024/tree/main/MOGE2025):

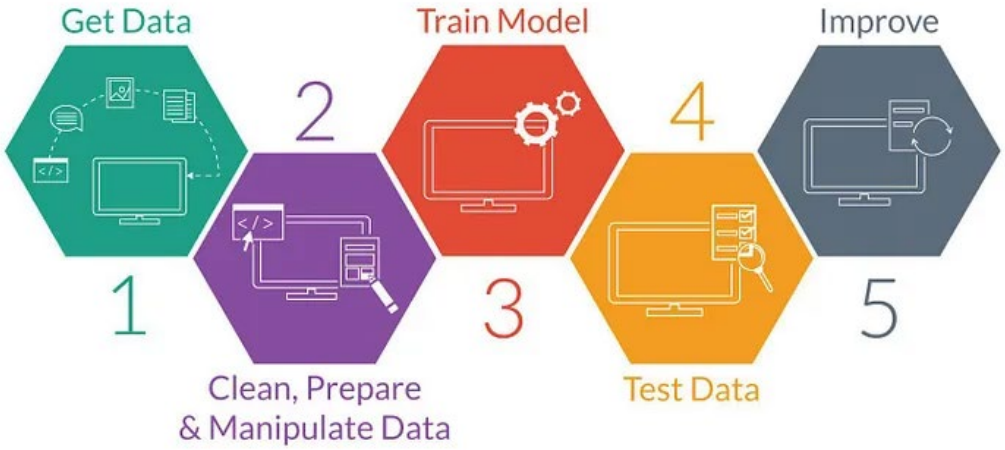
- **Introduction and motivation to scientific machine learning**
  1. Neural Networks and architectures
  2. Regression problems
- **Implementation of a classification problem in Grasshopper** (w/ Giovanni)

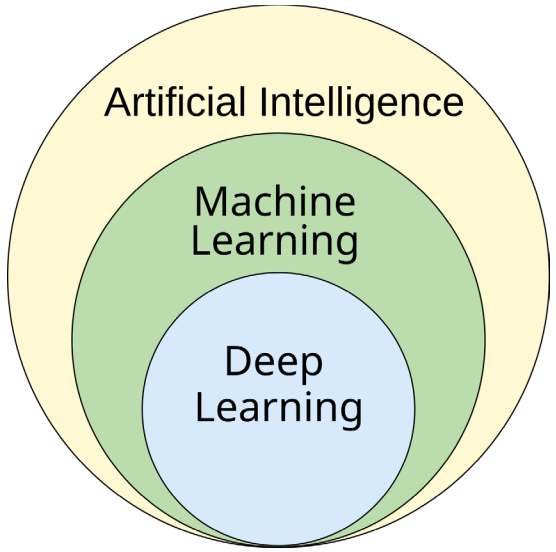
Learning goals:

- Understand basics of neural networks and their application
- Understand basics of regression problems
- Train your first neural network in Grasshopper

What is Machine Learning?

- Machine Learning concerns training computers to learn from data and make predictions.
- The term was coined in **1959** (!) by Arthur Samuel (IBM).
- Applications: healthcare (e.g., diagnosis), finance (e.g., algorithmic trading), transportation (e.g., autonomous vehicles), daily life (e.g., LLMs).



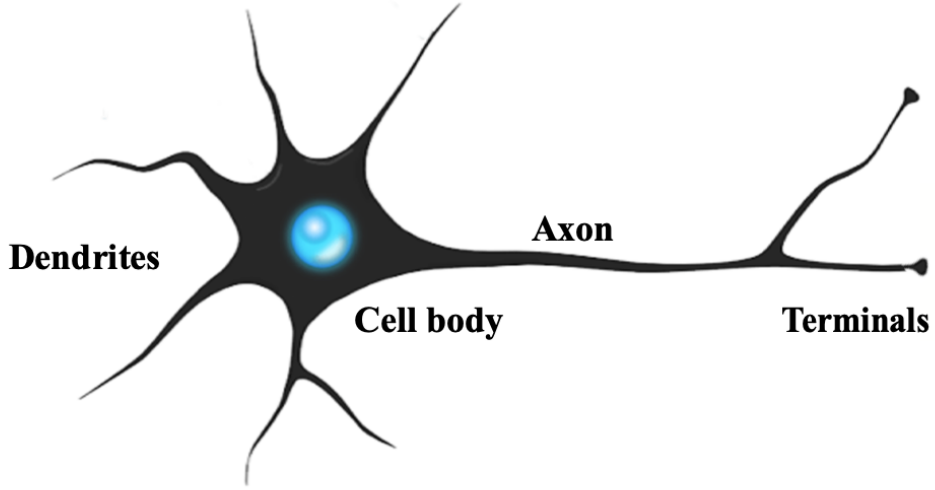


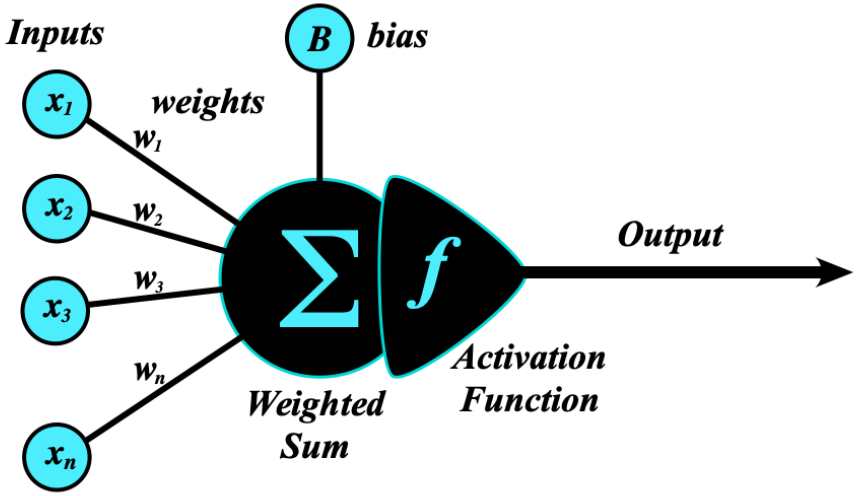
MODULE 1

INTRODUCTION TO NEURAL NETWORKS

Neural Networks - Neurons

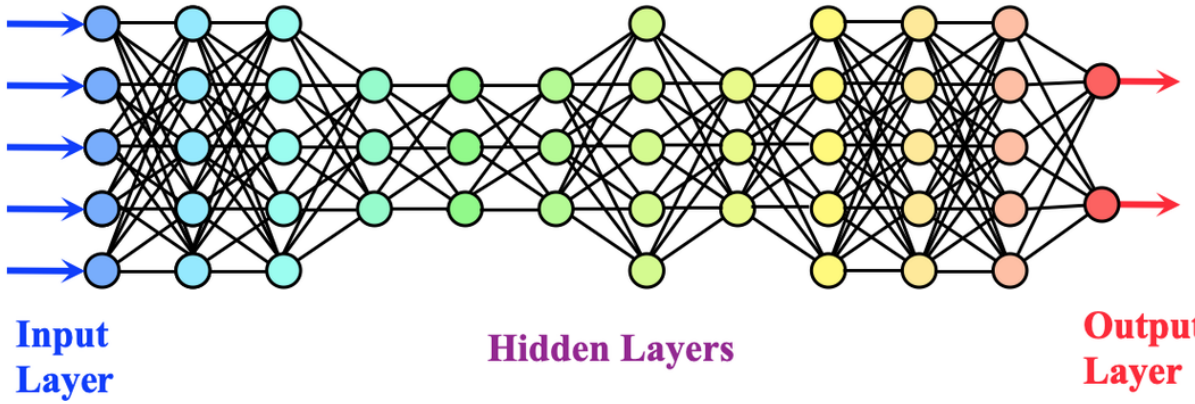
- **Neural networks** (NNs) are computational models inspired by the human brain's network of neurons.
- **Neurons** are the fundamental units of neural networks that process data.
- **Weights** determine the strength of the connection between neurons. They are adjusted during the training.
- **Activation functions** introduce non-linearity in the model.





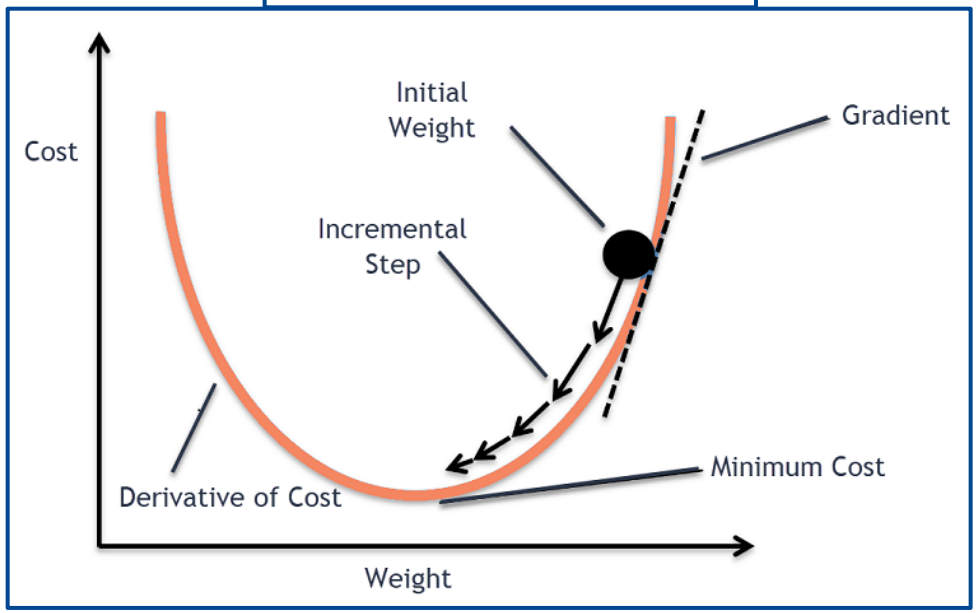
Neural Networks - Training

- NNs are composed of several **layers** of neurons.
- NNs takes an input and yield an output (**functions**)
- The training is performed via **backpropagation algorithm** .
- **Cost** function = (Output layer – training data)^2



Cost function

Gradient descent

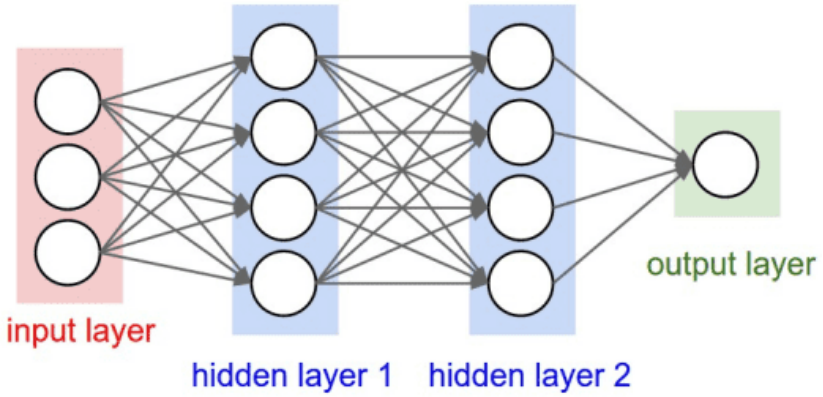


Optimal weights

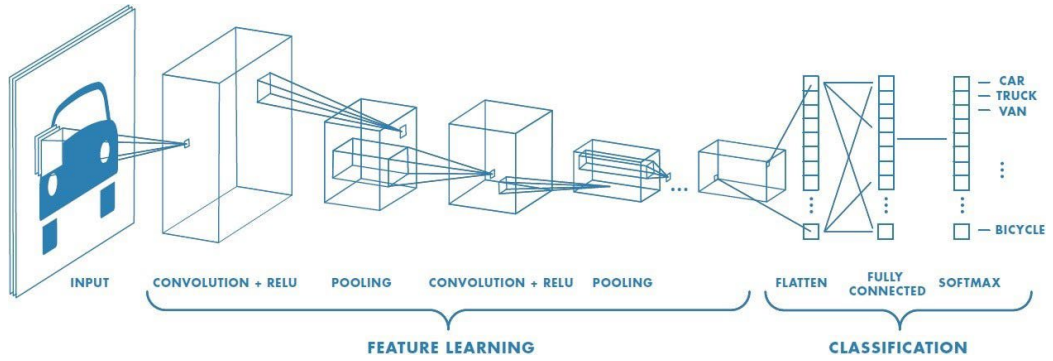
Neural Network Architectures (two examples)

- Different arrangements of neurons are called **architectures** .
- Different architectures are designed to be optimal for **different tasks** .

- **Feedforward Neural Network** :
  - Fully-connected layers (“perceptron”)
  - Used for classification and regression tasks
  - Can approximate functions (!!!)

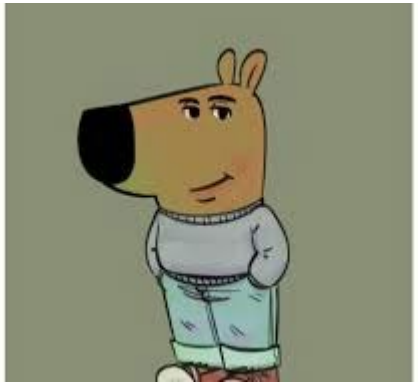


- **Convolutional Neural Network** :
  - Convolutional layers (filters)
  - Used for processing visual data
  - Used in facial recognition and medical images



What are they good at?

- **Real-world applications** :
  - Identification of objects, facial recognition, medical diagnostics, autonomous vehicles.
- **Natural Language Processing** :
  - Understand human language, chatbots, language translation, sentiment analysis
- **Applications in architecture** :
  - Generative design, structural analysis, building information modelling (BIM)
- **Applications in acoustics** :
  - Acoustic feature recognition, shape optimization, and more on the second module…



What are they bad at?

- Neural networks often require **large amounts of high-quality data**
- NNs require significant **computational power** (bad for the environment and time-consuming)
- Overfitting: the model cannot generalise and **perform poorly on unseen data**.
- NNs are **difficult to interpret** and are usually seen as “black boxes”.

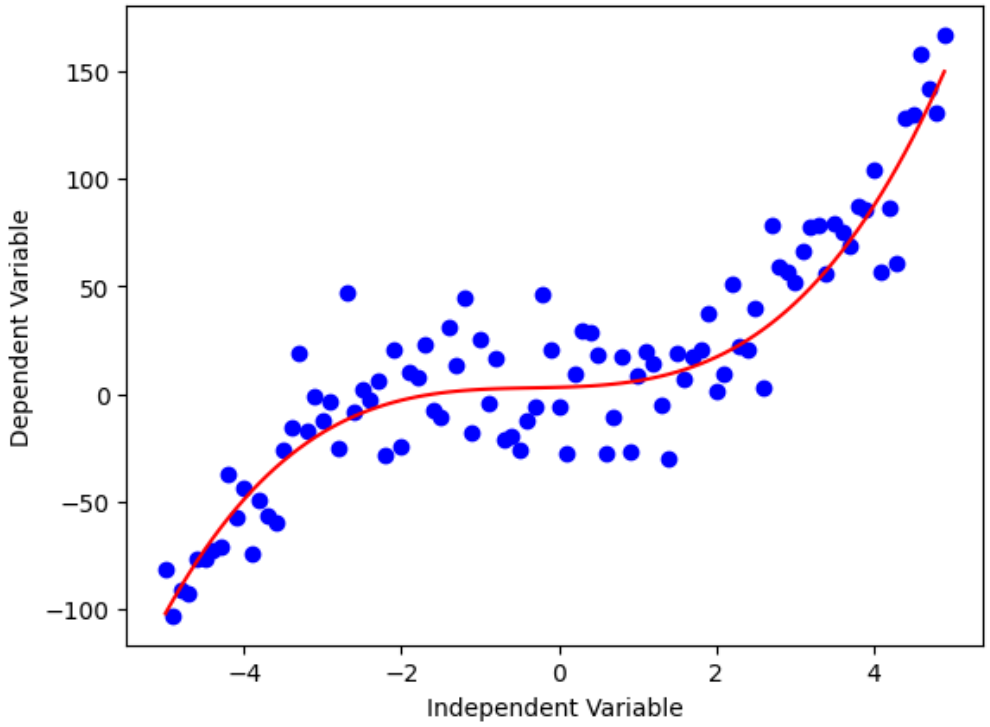
MODULE 2

INTRODUCTION TO REGRESSION PROBLEMS

What is a regression problem?

- **Regression** is a type of **supervised learning** task where the goal is to **predict a continuous (real -valued) output** from one or more input features.
- **Examples**: House price prediction, Weather forecasting, Energy demand estimation, Acoustic Reverberation Time.

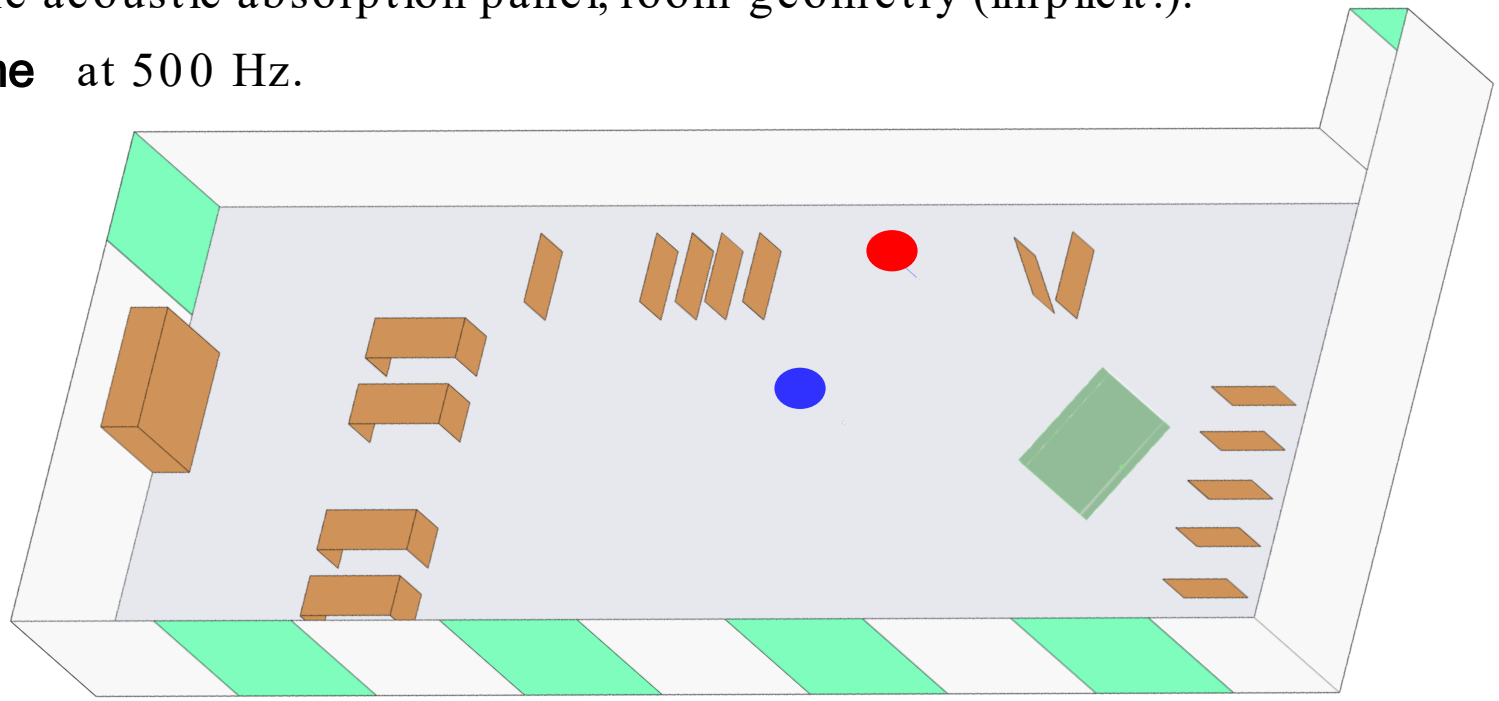
- **Neural networks** learn a continuous mapping from inputs to outputs by minimizing a suitable loss across a training set, then **generalize** to new data.
- **Example**: the network learns (from **past data**) how to associate temperature, humidity, pressure, wind speed to next day temperature. Then it generalizes to present and future data



Acoustic reverberation time

We will use a neural network to determine the **reverberation time** of the atelier upstairs, given the position of an **acoustic absorption panel** .

- Input: **position** (x,y) of the acoustic absorption panel; room geometry (implicit!).
- Output: **reverberation time** at 500 Hz.



- We will use a **Neural Network** to solve this problem!

1. Data pre-processing

The first step is to **load the dataset** . Usually, this step come after the production of the dataset. In our case it is pre-made by us (see GitHub repo).

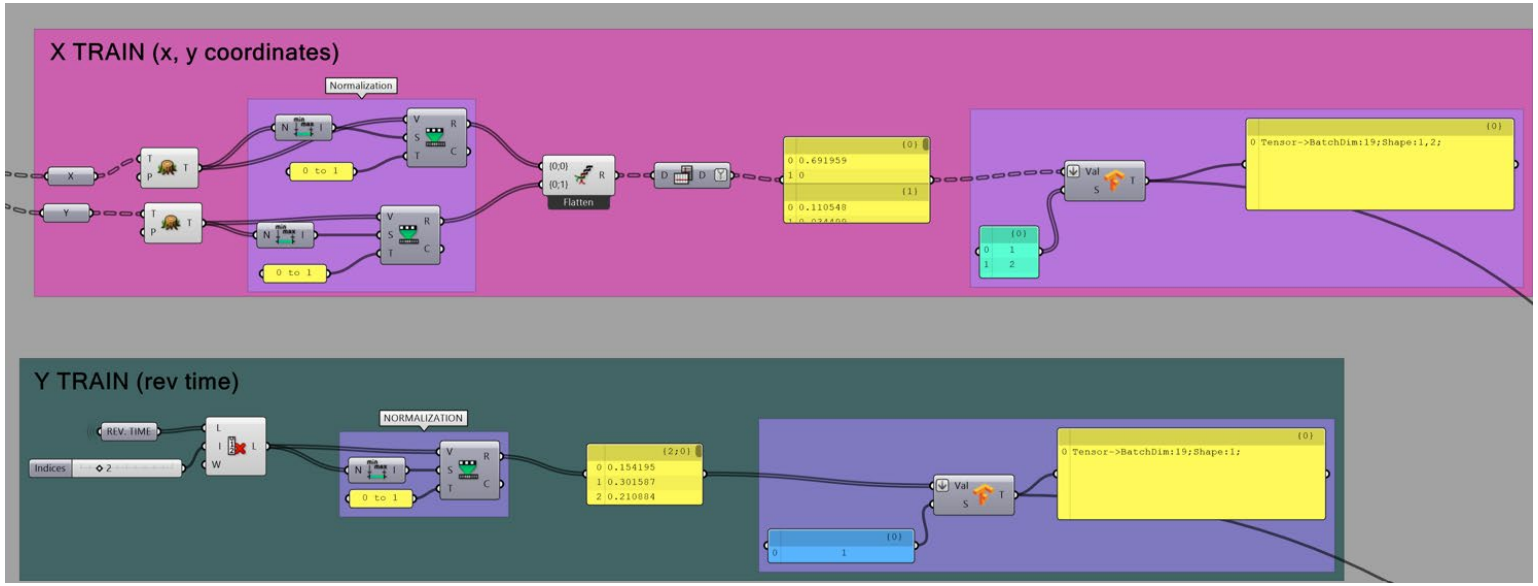
Third step is to **Normalize** the training dataset: all the values are in the range [0,1]

```
# scale into [0,1]
X_train_01 = (X_train - x_min) / (x_max - x_min)
X_val_01   = (X_val   - x_min) / (x_max - x_min)
X_test_01  = (X_test  - x_min) / (x_max - x_min)
```

The second step is to pre-process the training dataset:

➢**Split** the dataset into input and output.

```
# Training indices are all the rest:
train_idx = [i for i in range(len(X)) if i not in (val_idx, test_idx)]
X_train, y_train = X[train_idx], y[train_idx]
```



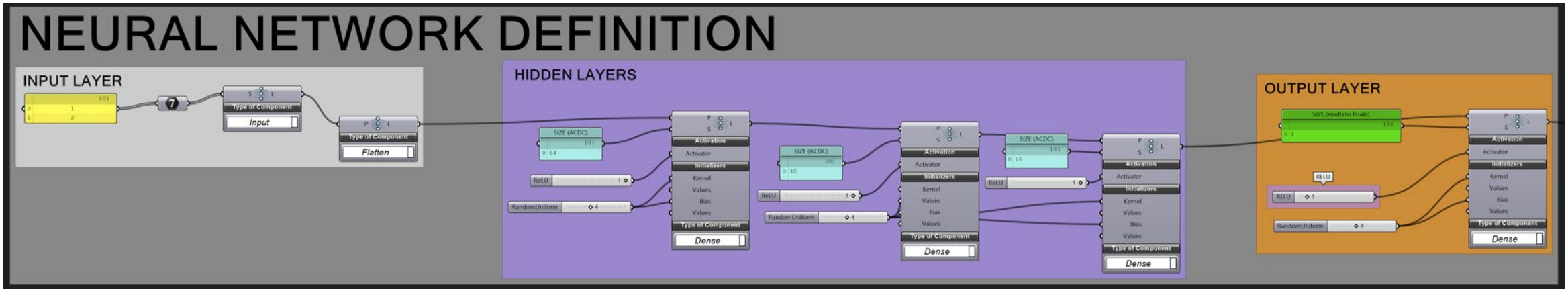
2. Define the neural network architecture

The next step is to build the **architecture of the neural network** .

The neural network has 5 layers:

- Input layer: 2 neurons (to receive x and y of panel position)
- Dense layer 1: 64 neurons
- Dense layer 2: 32 neurons
- Dense layer 3: 16 neurons
- Output layer: 1 neurons (reverberation time is one number)

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu',
    tf.keras.layers.Dense(32, activation='relu',
    tf.keras.layers.Dense(16, activation='relu',
    tf.keras.layers.Dense(1, activation='relu',
])
```

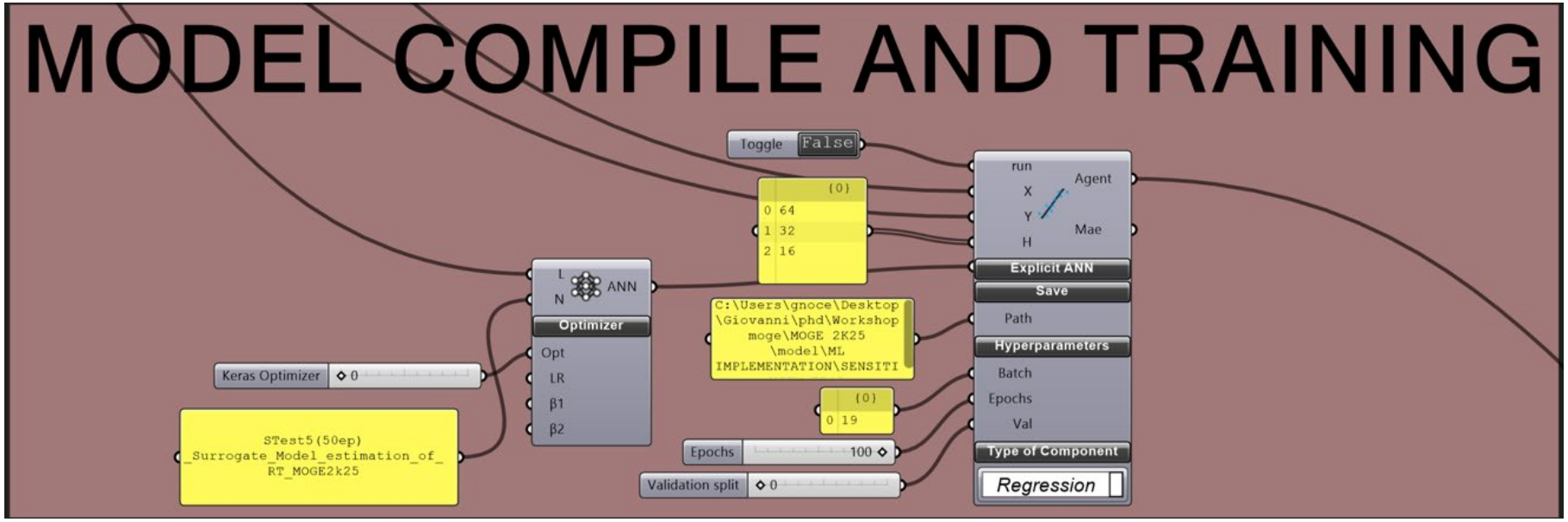


3. Compile the model

We need now to **compile the model** . This means that we must provide the following information:

```
# Compile the model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='mse')
```

- **Optimizer** : Adam (to solve the gradient descent)
- **Loss function** : mean square error
- **Learning rate** : 0.001

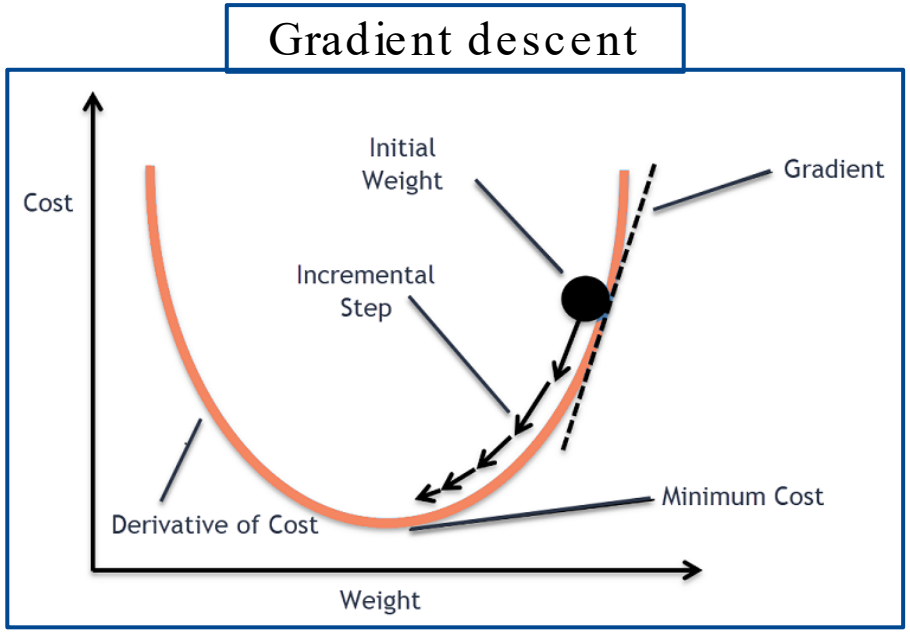


4. Train the neural network

The last step is to **train the neural network** .

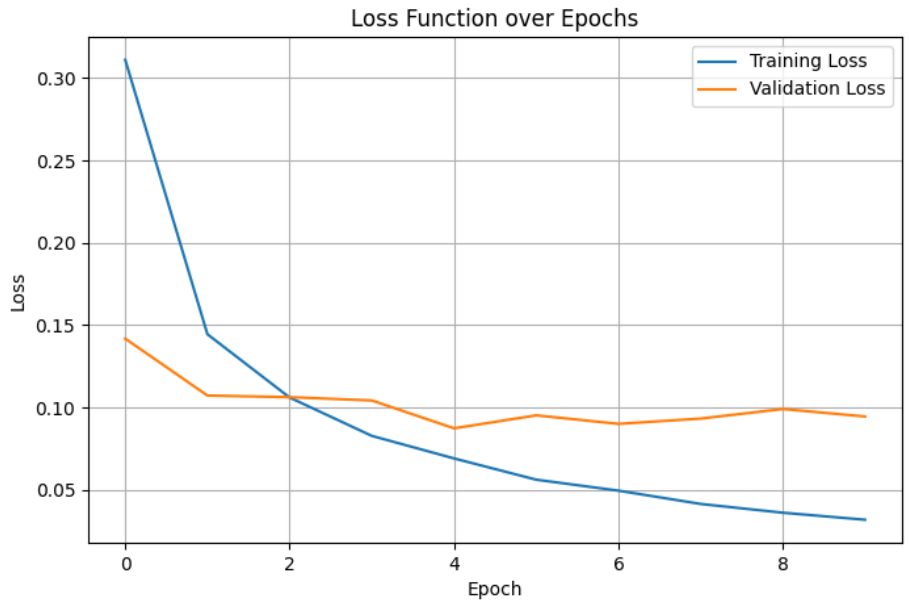
- We need to provide the following information:
  - **Inputs** (from pre-processing)
  - **Epochs**: how many times we must process the dataset
  - **Batch size**: how many instances provided at the same time

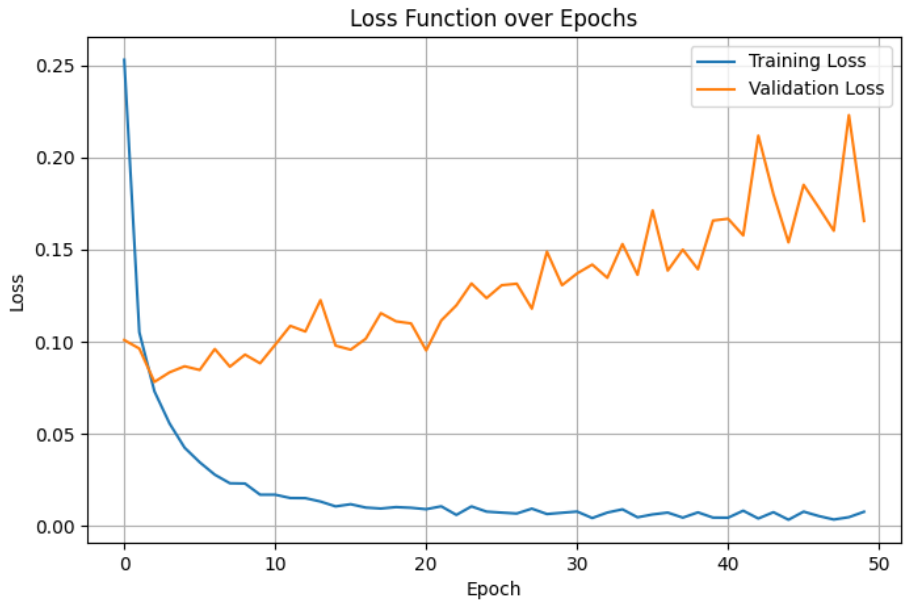
```
history = model.fit(
    X_train_01, y_train_01,
    validation_data=(X_val_01, y_val_01),
    epochs=500,
    callbacks=[es],
    verbose=0
)
```



Be aware of overfitting

- **Overfitting** occurs when a neural network learns the details and noise in the training data to such an extent that it **performs poorly on new, unseen data** .
- Instead of generalizing to capture the underlying patterns in the data, **the model becomes overly specialized to the training dataset** , resulting in high accuracy on the training data but poor performance on validation or test datasets.
- Overfitting can be caused by **training for too long** (too many epochs!)





Good luck with your implementation!

- Later you will perform the previous steps in Grasshopper to implement a neural network able to compute the reverberation time .
- Download this presentation from the repository and check the details of the implementation.

Thanks for your kind attention!

If you have additional questions, **ask me**, or send me an email at laudato@kth.se .

Ciao!